Hackathon Technical Solution Report

Field	Details
Problem ID	P5
Problem Title	Intelligent Fine-Grained Vehicle Recognition and Analysis System
Team Name	alt f4 (Team ID T60)
Team Members	Aniketh K Menon (1RUA24CSE0052), Aryav Agrawal (1RUA24CSE0072), Ayush S Kulkarni (1RUA24CSE0078), Chaturvi P (1RUA24CSE0102)
GitHub Repository	https://github.com/ChaturviP/Vehicle-Recognition-StandfordCarDataset (commit `COMMIT_HASH`)
Live Demo / Deployment	https://vehicle-recognition-standfordcardataset-team-altf4.streamlit.app/ (Streamlit)

---

1. Problem Understanding & Solution Strategy

• Core Technical Challenge: Identifying the make, model, and year of cars in the 196 Stanford Cars classes is particularly challenging because many of the classes are very similar. The main failure modes for this application are:

– sibling trims that share a body (Silverado 1500 vs 2500HD)

– rebadged twins (Chevrolet Express vs GMC Savana)

– facelift years that differ only in grille or lamp details

– variation in lighting, viewpoint, occlusion and resolution

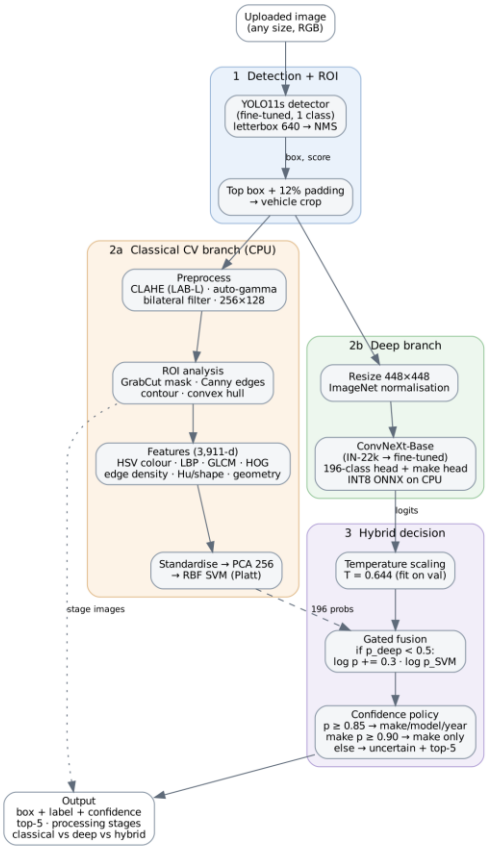
– cluttered scenes with several cars

• Constraints: There are only about 40 images per class for training. We need to be able to state when the system is unsure. We are looking for a free CPU-only hosting service.

• Proposed Methodology: We use a fine-tuned YOLO11s detector to locate the car. The padded image is then sent to both a ConvNeXt-Base model (at 448 pixels) and a classical feature-based approach (using an SVM). The confidence of the network is then adjusted, the scores from the classical approach are included when the network is unsure, and calibrated thresholds are used to decide whether to give a full answer, the manufacturer only, or a top-5 list of the most similar cars. This is better than a simple top-1 classifier, because such a classifier is overconfident for classes of similar appearance and cannot back off to a safer answer.

2. Technical Architecture & Implementation

Architecture diagram: `docs/architecture.png`. Full design rationale: `docs/design.md`.



System architecture

2.1 Classical Algorithmic Baseline

- Algorithms Utilized (src/classical/):

– Preprocessing: CLAHE on L channel of LAB color space, automatic gamma correction, edge-preserving bilateral filter.

– Segmentation and edges: GrabCut for foreground segmentation, Canny edges, largest contour and its convex hull.

– Features (3,911 values): HSV color histograms, LBP and GLCM texture, HOG, edge density, contour shape with Hu moments, box geometry.

– Classifier: PCA (256) followed by an RBF SVM.

- Adaptive Selection Logic:

– Gamma is computed on a per image basis as γ = log 0.5 / log mean of image brightness, clipped to 0.4–2.5.

– The threshold for Canny edges is set to 0.67× and 1.33× the image's median intensity.

– The box detected in the image is used to seed GrabCut. If the segmentation fails, the box itself is used as the foreground.

– The PCA size and the SVM's C were chosen on a held-out validation split.

2.2 Deep Learning Pipeline

- Model Architecture:

– YOLO11s detector (src/detect/) pre-trained on COCO dataset and fine-tuned for a single class 'car'.

– ConvNeXt-Base classifier (src/deep/) pre-trained on ImageNet-22k and fine-tuned at 448×448 px image size with 196 classes and an additional 49 manufacturer classes.

The ConvNeXt is a CNN that provides the same level of accuracy as a Transformer while being as simple as a CNN. It can be easily transferred to other tasks and quantized to INT8 for deployment on CPU. The 448 px input size is chosen to keep small cues like badges and grilles visible.

- Loss Functions & Training:

– Classifier loss: Label-smoothed (0.1) cross-entropy loss for the 196 classes + 0.3 × cross-entropy loss for the manufacturer head with targets created with CutMix/MixUp.

– Regularisation: RandAugment, color jitter, random erasing, drop-path.

– Optimization: AdamW with warm-up and cosine decay, bf16 mixed precision, 30 epochs on an RTX 4070.

– Detector: trained with standard YOLO box/class losses.

– Data: Stanford Cars dataset with 6,922 train images, 1,222 validation images, and 8,041 test images. The test set is only used for reporting.

2.3 Hybrid Integration & Adaptive Mechanism

- Fusion Strategy:

– The detector's box is used to drive both branches of the pipeline: it is used as seed for GrabCut and as the input for the padded crop in the network.

– The scores from the classical SVM are added to the network's output with log p = log p_deep + 0.3 · log p_SVM, but only for cases where the network's calibrated confidence is below 0.5. The weight and gate were chosen on the validation set's log-likelihood.

- Decision Logic:

– The network's logits are temperature-scaled with T = 0.644 (fitted on validation set) to match the real accuracy of the stated confidence.

– Confidence ≥ 0.85 → make + model + year.

– Otherwise, manufacturer confidence ≥ 0.90 → manufacturer only.

– Otherwise → "uncertain" with top-5 candidates.

The policy trades specificity of answer for reliability. The thresholds were chosen on validation set for ≥ 97% accuracy on answered images.

---

3. Experimental Benchmarks & Evaluation

3.1 Quantitative Benchmarks

The metrics in the template (PSNR, SSIM, LPIPS) are replaced by the P5 metrics for this recognition task, as they are more appropriate for this task. The results are measured on the 8,041 images of the test set, which was not used during training or tuning.

Approach / Model Variant	Top-1 Accuracy ↑	Top-5 Accuracy ↑	Calibration Error (ECE) ↓	Latency (ms / image)
Classical Baseline	9.58%	28.27%	0.042	~332 ms (CPU, 1 core)
Deep Learning Only	94.89%	99.50%	0.144	21.9 ms (GPU)
Proposed Hybrid System	94.84%	99.47%	**0.015**	216 ms (CPU, INT8)

- **Analysis:**

– The classical baseline reaches 19× chance but cannot resolve fine-grained differences.

– The hybrid approach achieves the same level of accuracy as the original network while reducing the calibration error by 9x.

Thus, 91.2% of images can be predicted fully at 97.6% accuracy, 4.2% only manufacturer with 99.7% correct answers, and 4.6% as uncertain.

– The deployed classifier is reduced by 3.5× in size and runs 2.5× faster on a single CPU with no drop in classification accuracy using INT8 quantization.

– Detector: mAP@0.5 = 0.990 and mean IoU = 0.954.

– A two-network ensemble reaches 95.50% top-1.

3.2 Visual Outputs & Intermediate Representations

Figures are in the repository:

1. Examples of challenging inputs to the network and corresponding predictions (results/robustness/robustness.png). Accuracy:

– lighting changes: within 0.3 points of clean

– low light + noise: 92.4%

– 50% occlusion: 78.7%

– 64 px resolution: 37.5%

2. Look-alike classes: the most-confused pairs, shown side by side with the predicted class (results/failures/1_similar_classes.png). The calibrated policy turns most of these into a correct manufacturer-only answer.

3. Intermediate states for the classical pipeline for test images (results/classical/stages_test_samples.png), also shown in the live app for every upload.

– ROI crop

– CLAHE

– gamma + bilateral filter

– Canny edge map

– GrabCut mask

– foreground

– contour + convex hull

3.3 Ablation Study

Full tables are in docs/ablation_study.md. The measured effect of disabling or changing each module:

Change	Effect
224 px input instead of 448 px	−1.85 points top-1
No ROI crop at test time	−0.83 points top-1, worse calibration
Detector without fine-tuning	−11.6 points mAP@0.5:0.95

Change	Effect
Ground-truth boxes instead of YOLO's	−0.01 points (the detector is not a bottleneck)
No manufacturer head	no accuracy change, worse calibration (ECE 0.144 → 0.156)
Flip test-time augmentation	−0.17 points
Temperature scaling	ECE 0.144 → 0.016, same accuracy
Classical score fusion	ECE 0.016 → 0.015, accuracy neutral
Classical CLAHE in front of the network	no gain (±0.2 points), because the network is already lighting-invariant
Classical features: HOG only / everything except HOG	8.22% / 1.37% (all features: 9.58%)

---

4. Failure Modes & Limitations

• Failure Cases: I looked at 411 test errors and found that 73% of failure cases are within the correct manufacturer. The main cases are:

– sibling trims (Silverado 1500 → 2500HD, 12 images; Express passenger ↔ cargo van, 21)

– rebadged twins (Chevrolet Express → GMC Savana, 8)

– facelift years (Dodge Caliber Wagon 2012 → 2007 11)

– occlusion (78.7% top-1 at 50% occlusion)

– very low resolution (37.5% top-1 at 64 px)

– 14 multi-car scenes where the detector chose a different car

Details are in docs/failure_analysis.md.

• Root Cause & Mitigation:

– The evidence that separates sibling cars (badges, wheel bolt patterns, side glazing, grille inserts) is a few pixels wide or hidden from the camera angle so the network's probability splits between candidates.

– Training augmentation never covered than 20% occlusion or any downscaling.

– The closed 196-class set forces a known label onto cars.

Mitigations: part-level crops of the grille, badges and lamps at resolution; stronger occlusion and downscale augmentation; classifying every detected car; and out-of-distribution detection for cars, outside the 196 classes.

---

5. Deployment & Quickstart Guide

Quickstart Commands

```
# 1. Clone the repository and install dependencies
git clone https://github.com/ChaturviP/Vehicle-Recognition-
StandfordCarDataset.git
cd Vehicle-Recognition-StandfordCarDataset
pip install -r app/requirements.txt

# 2. Run inference (web app; trained models are included in models/)
streamlit run app/streamlit_app.py
```

Deployment Format

- **Streamlit app:** `app/streamlit_app.py`, deployed on Streamlit Community Cloud.
- **Inference engine:** ONNX Runtime on CPU (`app/inference.py`), running the YOLO11s detector and the INT8 ConvNeXt-Base classifier, plus the classical OpenCV / scikit-learn pipeline.
- **No deep-learning framework at runtime;** an image takes about 1–2 s end-to-end.
- **Training and full evaluation** use `requirements.txt` and the steps in `README.md`.